Chapter 3: Processes

Chapter 3 Processes

3.1 Process Concept 106
3.2 Process Scheduling 110
3.3 Operations on Processes 116
3.4 Interprocess Communication 123
3.5 IPC in Shared-Memory Systems 125
3.6 IPC in Message-Passing Systems 127
3.7 Examples of IPC Systems 132
3.8 Communication in Client–Server Systems 145
3.9 Summary 153
Practice Exercises 154
Further Reading 156

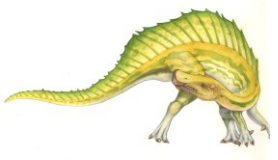
Coverage: 3.1 (Diagrams), 3.2 (diagrams), 3.3 (C coding), 3.4 (theory), 3.5 (C coding only), 3.7.1 (C coding only), 3.7.4 (C coding only), 3.8.2 (basic theory only)

Silberschatz, Galvin and Gagne ©2018



Objectives

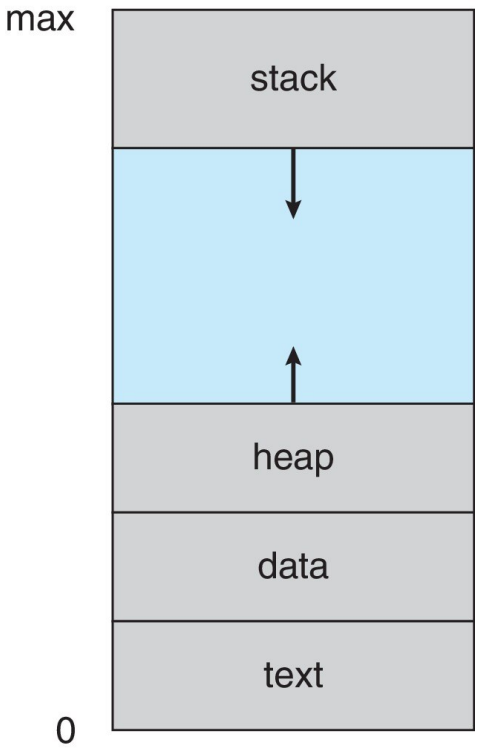
- Identify the separate components of a process and illustrate how they are represented and scheduled in an operating system.
- Describe how processes are created and terminated in an operating system, including developing programs using the appropriate system calls that perform these operations.
- Describe and contrast interprocess communication using shared memory and message passing.
- Design programs that uses pipes and POSIX shared memory to perform interprocess communication.
- Design kernel modules that interact with the Linux operating system.





Process Concept

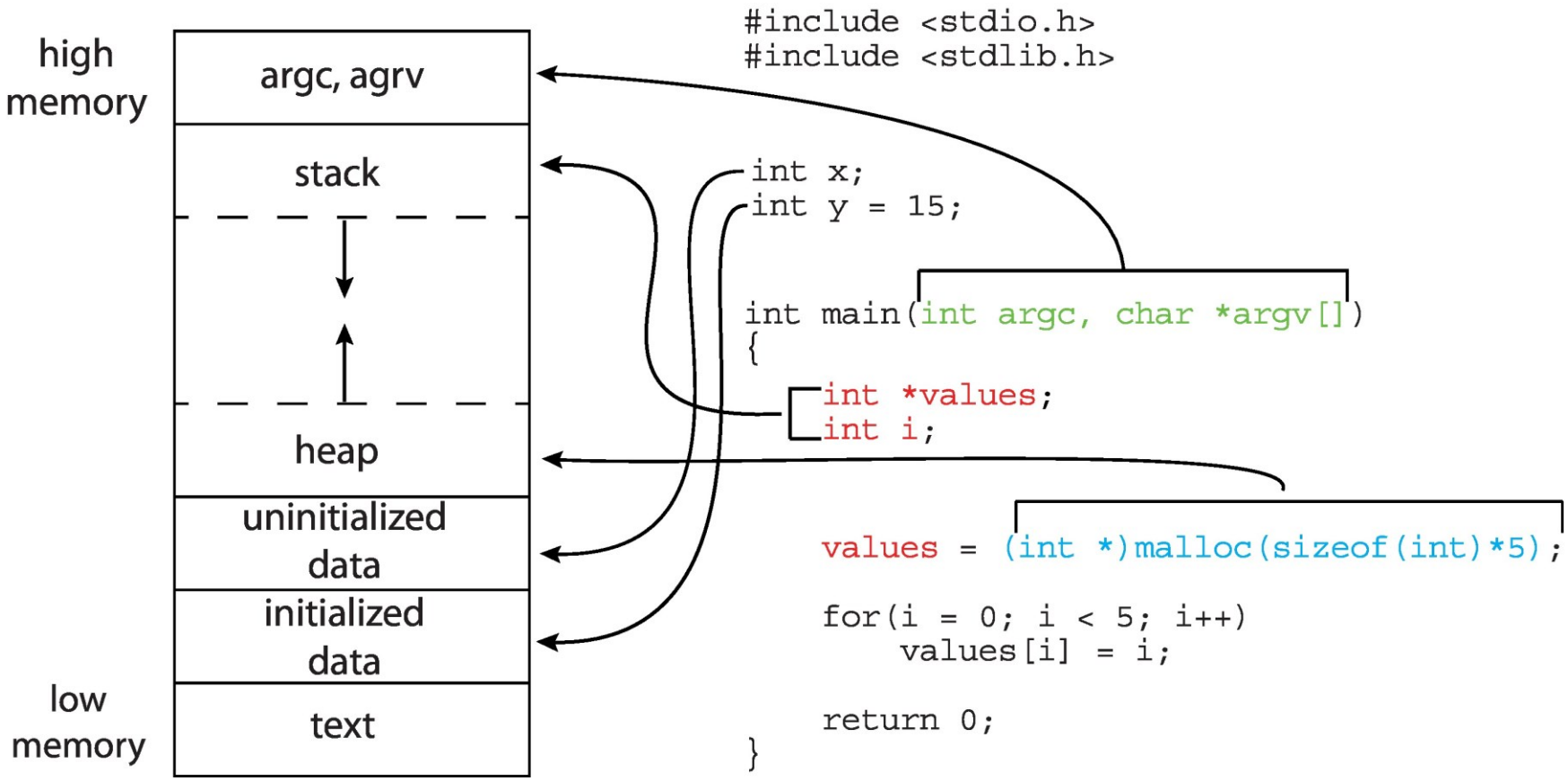
- An operating system executes a variety of programs that run as a process.
- **Process** – a program in execution; process execution must progress in sequential fashion. No parallel execution of instructions of a single process
- Multiple parts
  - The program code, also called **text section**
  - Current activity including **program counter**, processor registers
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Data section** containing global variables
  - **Heap** containing memory dynamically allocated during run time
- Program is **passive** entity stored on disk (**executable file**); process is **active**
  - Program becomes process when an executable file is loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can be several processes
  - Consider multiple users executing the same program

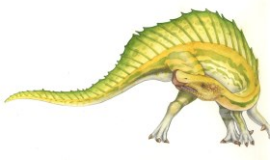






Memory Layout of a C Program

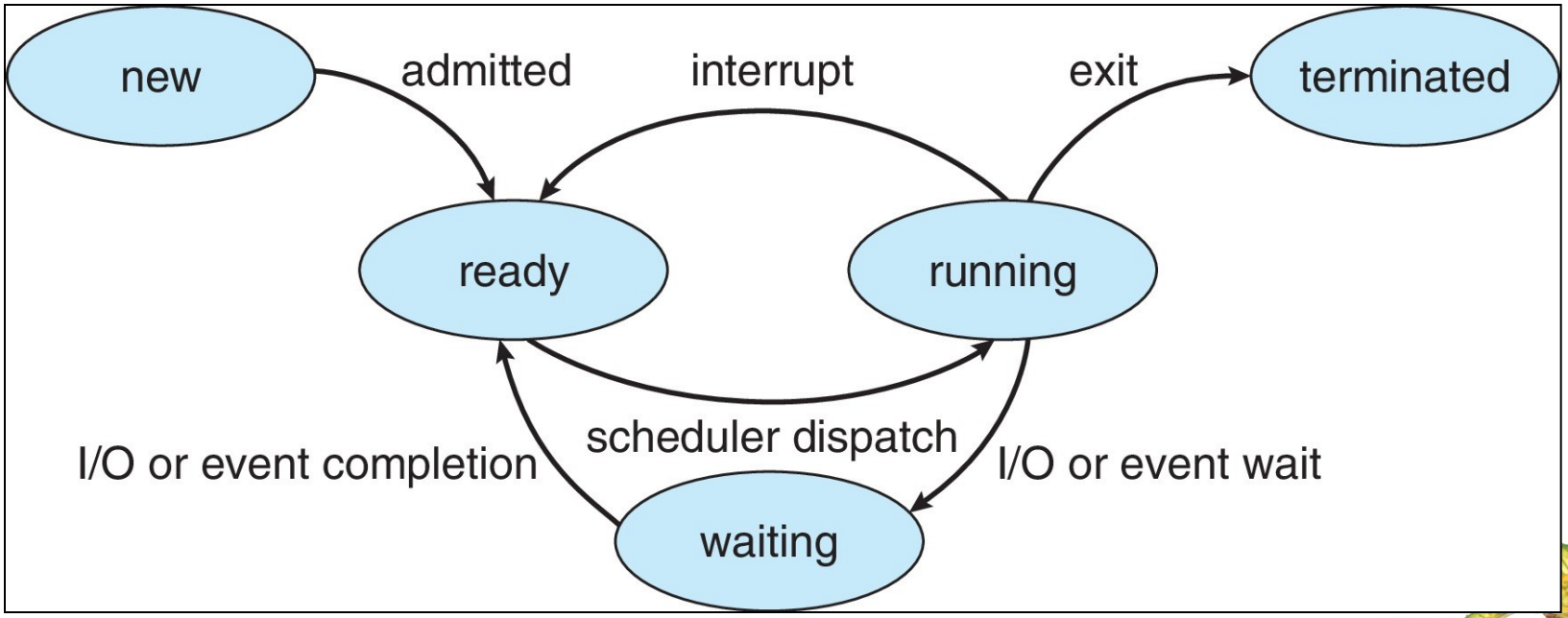






Process State

- As a process executes, it changes **state**
  - **New**: The process is being created
  - **Running**: Instructions are being executed
  - **Waiting**: The process is waiting for some event to occur
  - **Ready**: The process is waiting to be assigned to a processor
  - **Terminated**: The process has finished execution



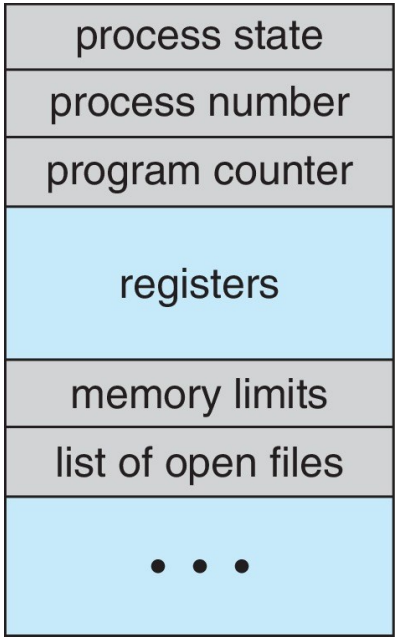


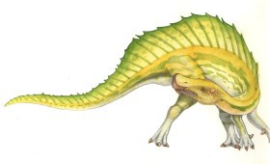


Process Control Block (PCB)

Information associated with each process(also called **task control block**)

- Process state – running, waiting, etc.
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

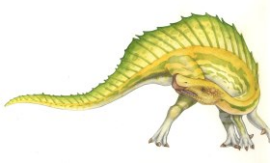






Threads

- So far, process has a single thread of execution
- Consider having multiple program counters per process
  - Multiple locations can execute at once
    - Multiple threads of control -> **threads**
- Must then have storage for thread details, multiple program counters in PCB
- Explore in detail in Chapter 4

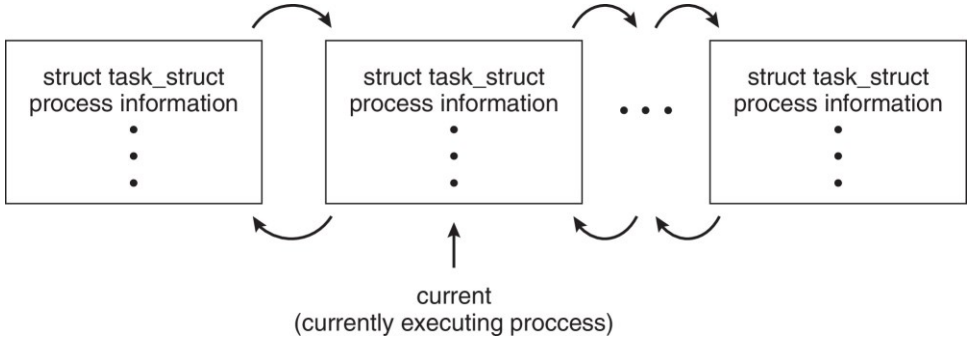


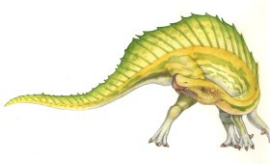


Process Representation in Linux

Represented by the C structure `task_struct`

```
pid t_pid;                    /* process identifier */
long state;                   /* state of the process */
unsigned int time_slice       /* scheduling information */
struct task_struct *parent;/* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files;/* list of open files */
struct mm_struct *mm;         /* address space of this
process */
```





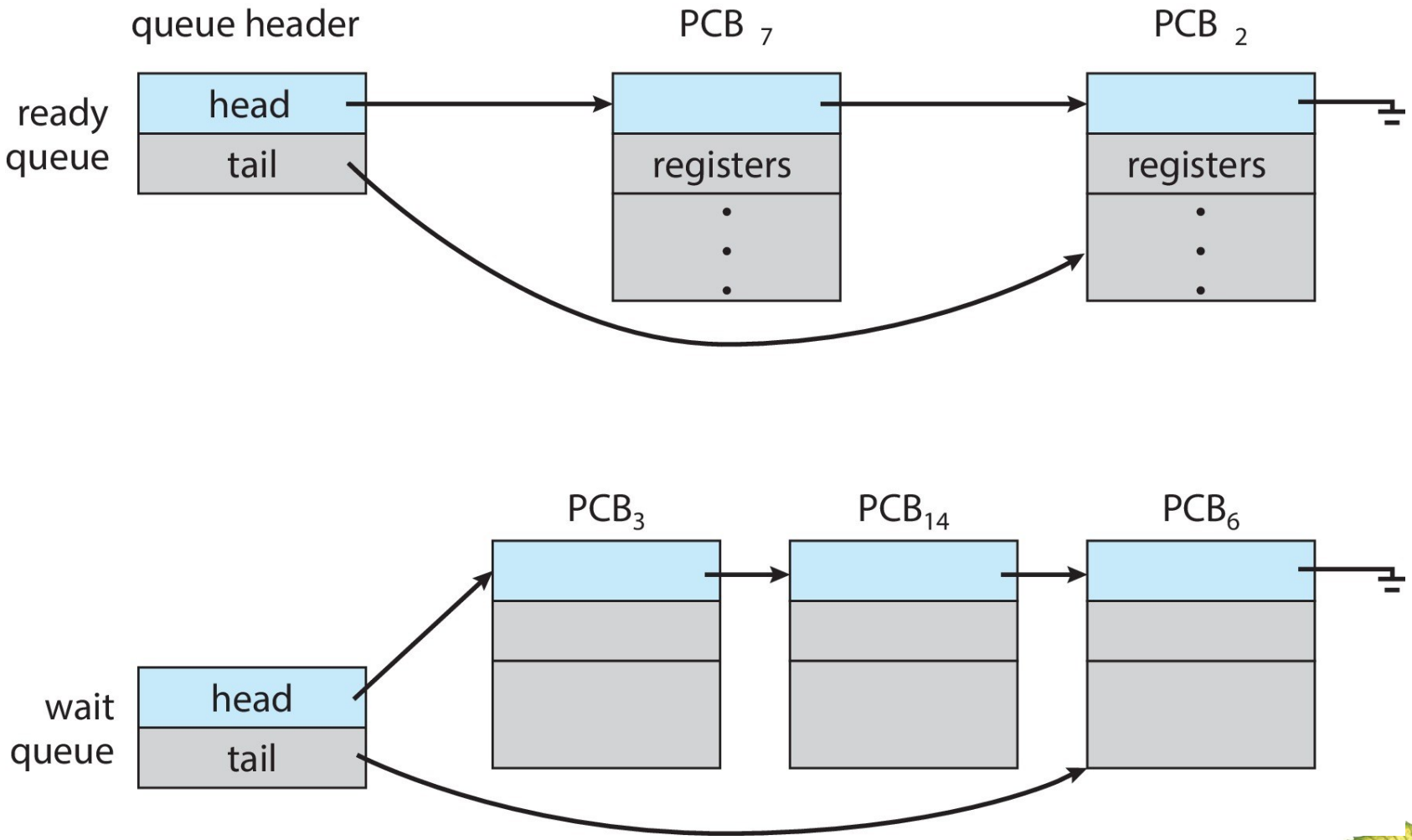


Process Scheduling

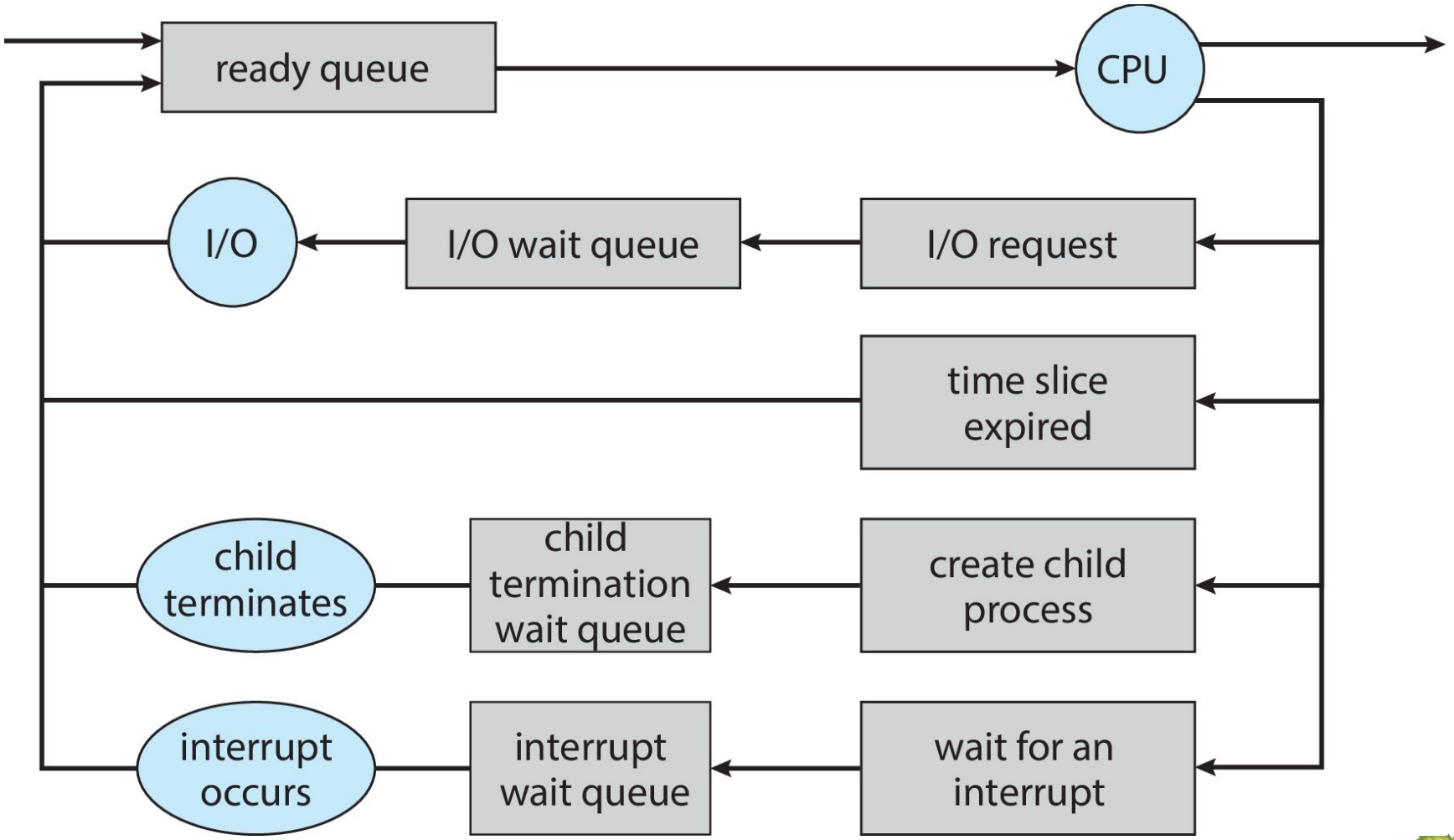
- **Process scheduler** selects among available processes for next execution on CPU core
- Goal -- Maximize CPU use, quickly switch processes onto CPU core
- Maintains **scheduling queues** of processes
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Wait queues** – set of processes waiting for an event (i.e., I/O)
  - Processes migrate among the various queues



Ready and Wait Queues



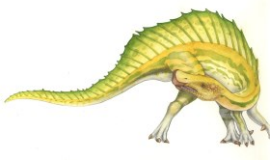
Representation of Process Scheduling





Week # 4 – Lecture # 2 & # 3

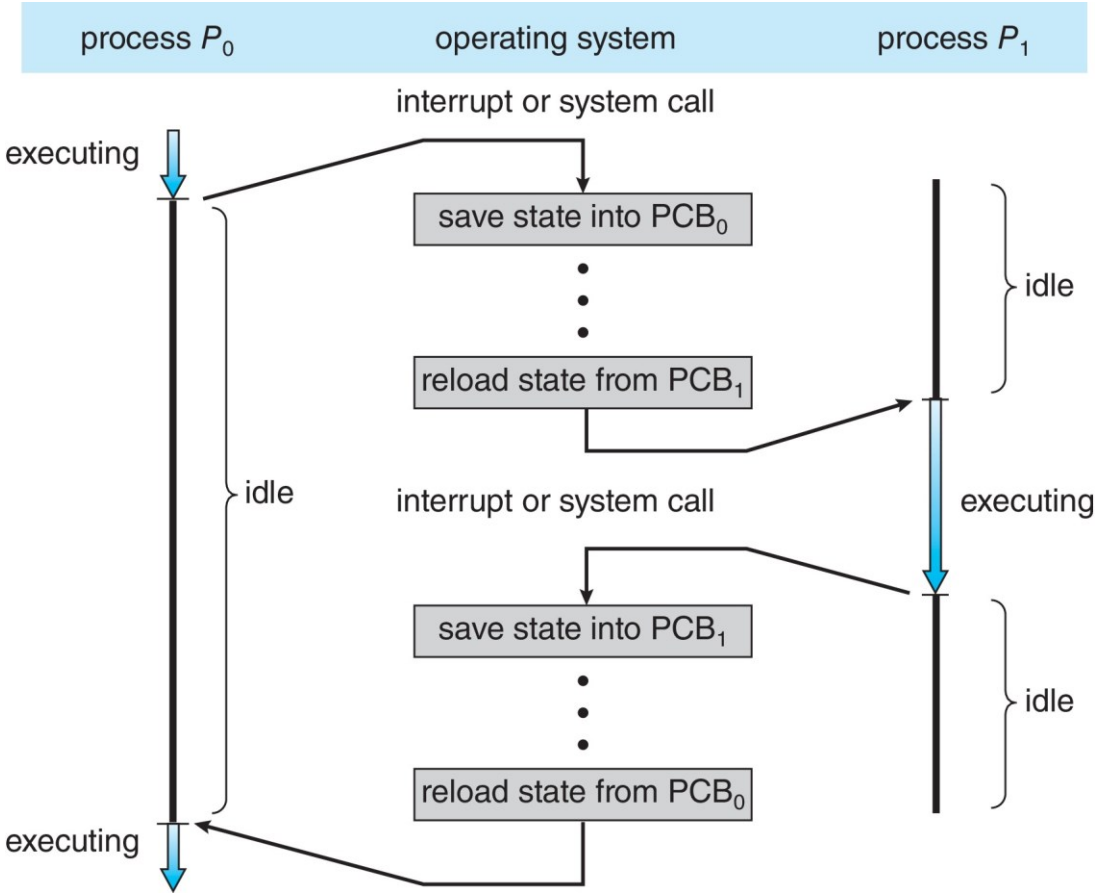
- Context Switching
- Processing creation:
  - fork(), execlp()
- Process Termination
- Inter-process Communication (IPC)
  - Shared Memory
  - Message Passing
- Producer Consumer Problem
  - Definition and link to IPC
  - Producer and Consumer Code
  - Challenges and short coming

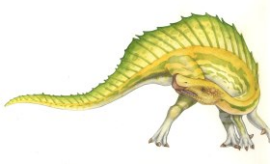




CPU Switch From Process to Process

A **context switch** occurs when the CPU switches from one process to another.

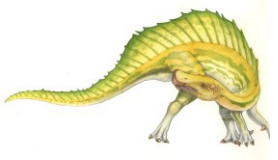






Context Switch

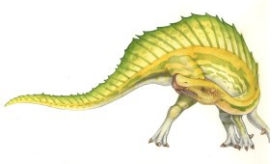
- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- **Context-switch time is pure overhead**; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once





Multitasking in Mobile Systems

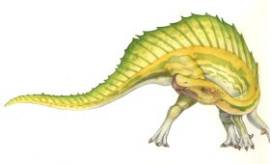
- Some mobile systems (e.g., early version of iOS) allow only one process to run, others suspended
- Due to screen real estate, user interface limits iOS provides for a
  - Single **foreground** process- controlled via user interface
  - Multiple **background** processes– in memory, running, but not on the display, and with limits
  - Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback
- Android runs foreground and background, with fewer limits
  - Background process uses a **service** to perform tasks
  - Service can keep running even if background process is suspended
  - Service has no user interface, small memory use





Operations on Processes

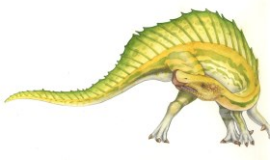
- System must provide mechanisms for:
  - Process creation
  - Process termination





Process Creation

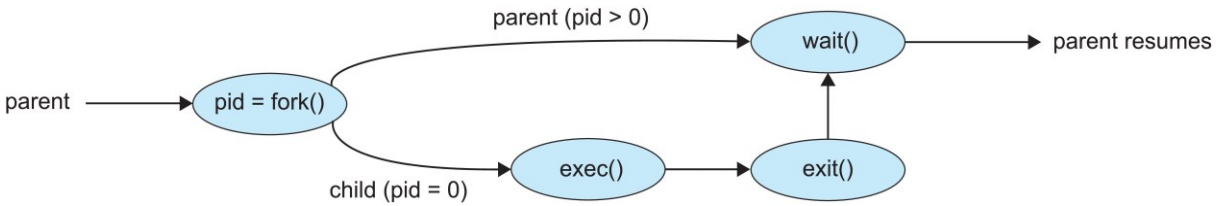
- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier** (**pid**)
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate

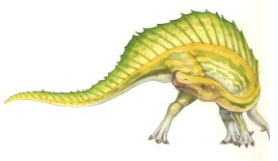




Process Creation (Cont.)

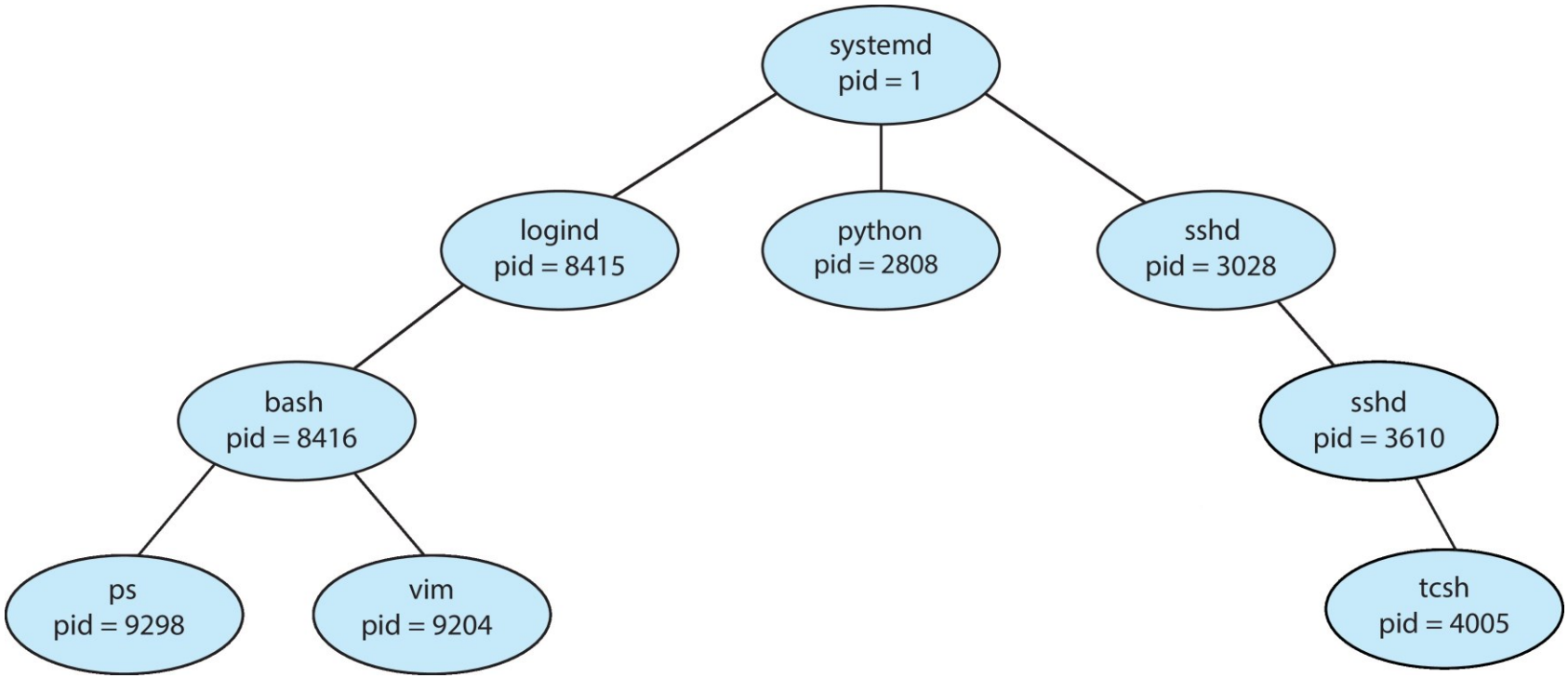
- Address space
  - Child duplicate of parent
  - Child has a program loaded into it
- UNIX examples
  - **`fork()`** system call creates new process
  - **`exec()`** system call used after a **`fork()`** to replace the process' memory space with a new program
  - Parent process calls **`wait()`** waiting for the child to terminate

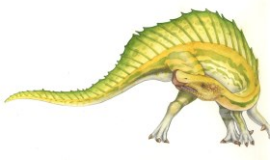






A Tree of Processes in Linux







C Program Forking Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

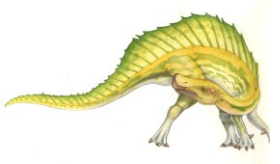
    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

```
Parent (PID X)
└── Child (PID Y) → Executes `ls` command
```

Where:

- **X** = Parent process PID.
- **Y** = Child process PID created by `fork()`.





Dry Run Analysis

1. **Parent process starts execution.**
2. `fork()` is called:
   - **Parent Process (PID X)** continues.
   - **Child Process (PID Y)** is created.
3. **Child Process** executes `execlp("/bin/ls", "ls", NULL)`:
   - Replaces its memory image with the `ls` program.
   - Executes `ls` (lists the directory contents).
4. **Parent Process** waits using `wait(NULL)` for **Child Process** to complete.
5. **Child Process** completes the execution of `ls` and terminates.
6. **Parent Process** resumes and prints:





C Program – execlp () details

Key Features of `execlp`

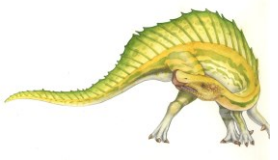
1. **Search in PATH**:
   - The `execlp` function searches for the executable file in the directories listed in the `PATH` environment variable.
   - This makes it convenient for executing system commands or programs without specifying the full path.
2. **Argument List**:
   - It takes a variable number of arguments:
     - The first argument is the filename of the executable.
     - The second argument is typically the name of the program (same as the filename or a custom name).
     - Subsequent arguments are passed as command-line arguments to the new program.
     - The argument list must end with a `NULL` pointer.
3. **No Return on Success**:
   - If `execlp` succeeds, it does not return, as the current process image is replaced.
   - If it fails, it returns `-1` and sets `errno` to indicate the error.





Process Termination

- Process executes last statement and then asks the operating system to delete it using the **`exit()`** system call.
  - Returns status data from child to parent (via **`wait()`**)
  - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **`abort()`** system call. Some reasons for doing so:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting, and the operating systems does not allow a child to continue if its parent terminates



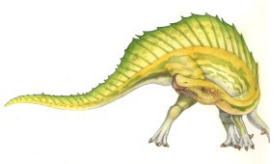


Process Termination

- Some operating systems do not allow child to exists if its parent has terminated. If a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc., are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **`wait()`** system call. The call returns status information and the pid of the terminated process

```
pid = wait(&status);
```

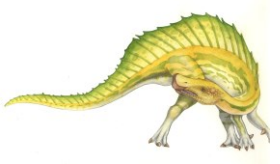
- If no parent waiting (did not invoke **`wait()`**) process is a **zombie**
- If parent terminated without invoking **`wait()`**, process is an **orphan**





Android Process Importance Hierarchy

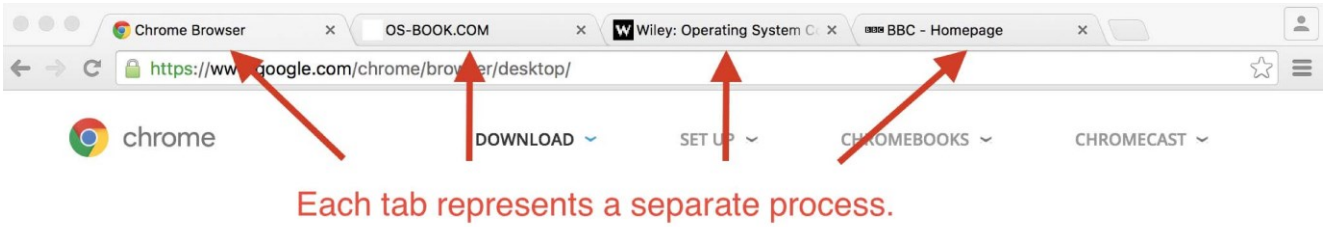
- Mobile operating systems often have to terminate processes to reclaim system resources such as memory. From **most** to **least** important:
  - Foreground process
  - Visible process
  - Service process
  - Background process
  - Empty process
- Android will begin terminating processes that are least important.

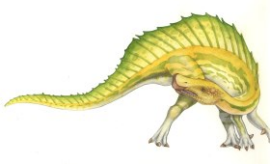




Multiprocess Architecture – Chrome Browser

- Many web browsers ran as single process (some still do)
  - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 different types of processes:
  - **Browser** process manages user interface, disk and network I/O
  - **Renderer** process renders web pages, deals with HTML, Javascript. A new renderer created for each website opened
    - Runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
  - **Plug-in** process for each type of plug-in







Interprocess Communication

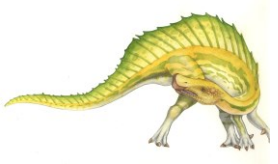
- Processes within a system may be ***independent*** or ***cooperating***
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**





IPC – Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.
- Synchronization is discussed in great details in Chapters 6 & 7.

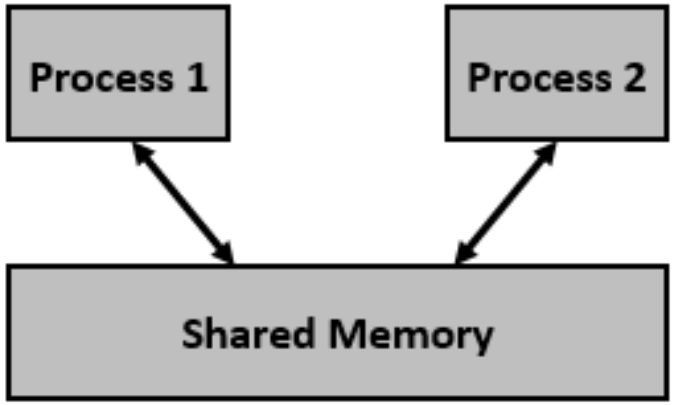


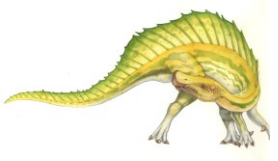


IPC – Shared Memory

Shared Memory IPC

- **Definition**: A form of inter-process communication (IPC) where multiple processes access a common region of memory. Processes can read and write directly to this shared memory area.
- **Characteristics**:
  - **Fast**: No kernel intervention required for data access.
  - **Synchronization Required**: Processes must use mechanisms like semaphores or mutexes to avoid race conditions.
  - **Direct Access**: Processes share the same memory space, enabling high-speed data exchange.



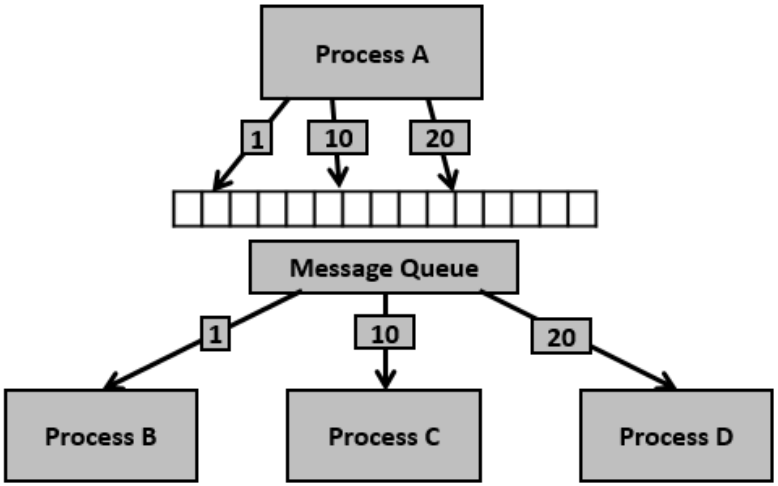


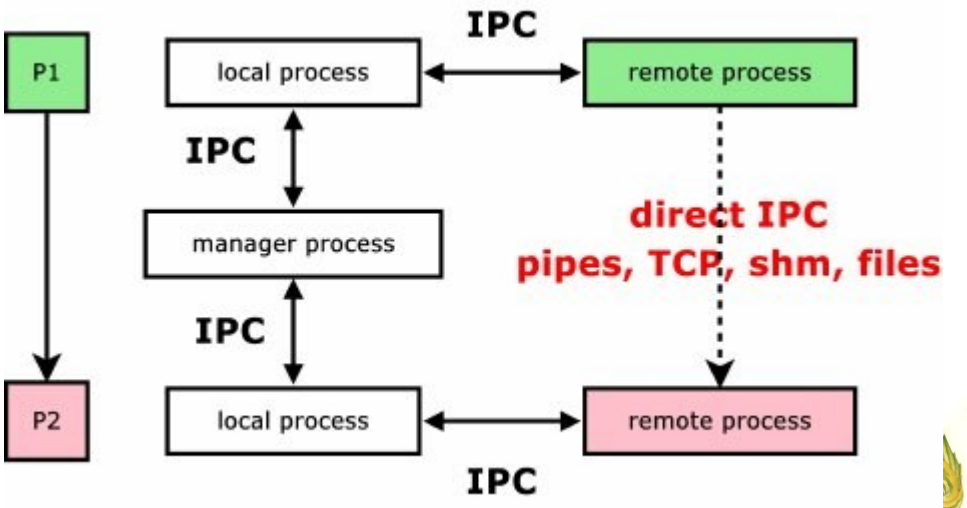


IPC – Message Passing

Message-Passing IPC

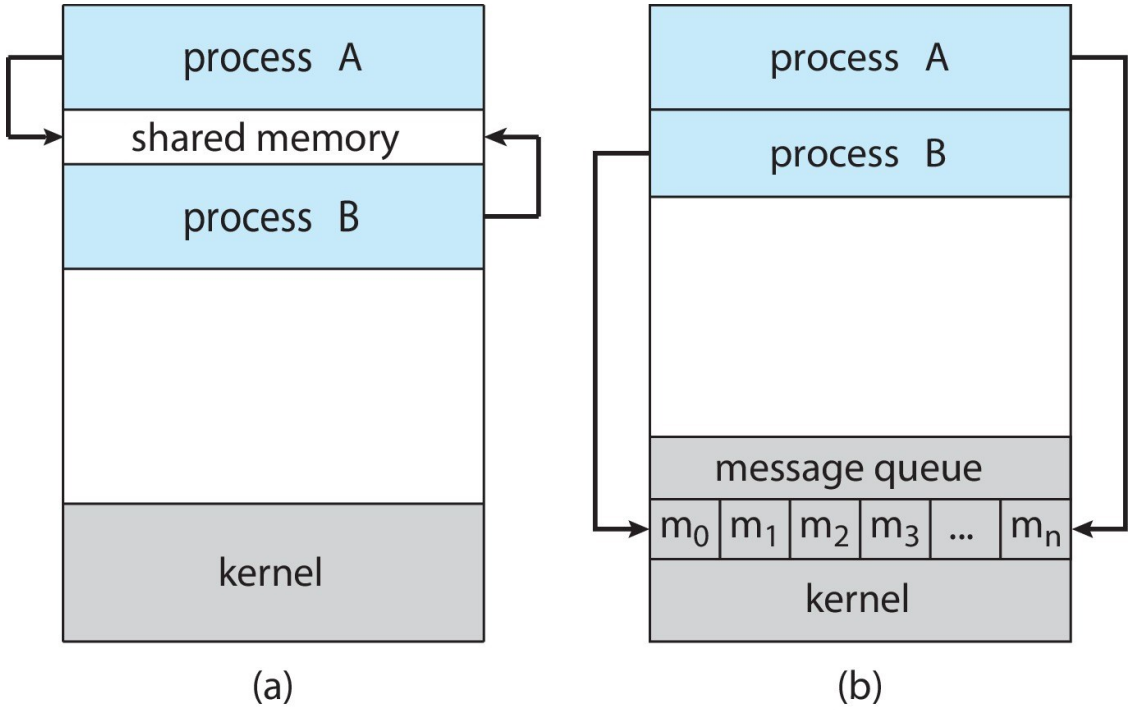
- **Definition**: A form of IPC where processes communicate by sending and receiving messages through a communication channel managed by the operating system.
- **Characteristics**:
  - **Kernel-Mediated**: Messages are passed via system calls, ensuring synchronization and data integrity.
  - **Slower**: Involves kernel overhead for message handling.
  - **Indirect Access**: Processes do not share memory; communication is explicit and structured.







Communications Models



(a) Shared memory. (b) Message passing.

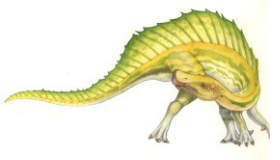
Key Difference

- **Shared Memory IPC**: Processes share a common memory region for direct, high-speed communication, requiring explicit synchronization.
- **Message-Passing IPC**: Processes communicate through kernel-managed messages, ensuring synchronization but with higher overhead.



IPC Types: Comparison

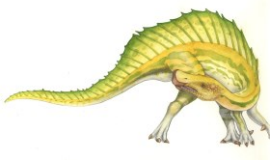
Aspect	Message Passing	Shared Memory
Communication Mechanism	Processes communicate by sending and receiving messages.	Processes communicate by reading and writing to a shared memory region.
Address Space	Processes have separate address spaces.	Processes share the same address space.
Synchronization	Built into `send()` and `receive()` operations.	Requires explicit synchronization (e.g., locks or semaphores).
Complexity	System-level implementation can be complex (e.g., variable-sized messages).	Easier system-level implementation but requires careful programming.
Use Cases	Ideal for distributed environments (e.g., Internet chat programs).	Best for systems where processes run on the same machine.





IPC Types: Contrast

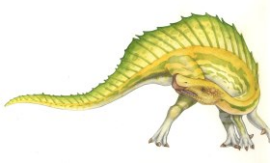
Aspect	Message Passing	Shared Memory
Memory Sharing	No shared memory; processes communicate through messages.	Relies on shared memory for communication.
Implementation Tradeoffs	The tradeoff between fixed-size (easier system-level) and variable-size messages (easier programming).	There is no such tradeoff; the focus is on managing access to shared memory.
Communication Link	Requires a logical communication link (direct/indirect, synchronous/asynchronous, buffering).	It does not require a separate communication link; processes directly access shared memory.





Producer-Consumer Problem

- Paradigm for cooperating processes:
  - *producer* process produces information that is consumed by a *consumer* process
- Two variations:
  - **unbounded-buffer** places no practical limit on the size of the buffer:
    - Producer never waits
    - Consumer waits if there is no buffer to consume
  - **bounded-buffer** assumes that there is a fixed buffer size
    - Producer must wait if all buffers are full
    - Consumer waits if there is no buffer to consume





Bounded-Buffer – Shared-Memory Solution

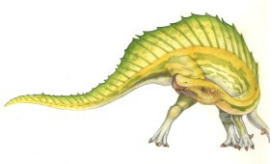
Let's look more closely at how the bounded buffer illustrates interprocess communication using shared memory. The following variables reside in a region of memory shared by the producer and consumer processes:

```
#define BUFFER_SIZE 10

typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

The shared `buffer` is implemented as a circular array with two logical pointers: `in` and `out`. The variable `in` points to the next free position in the buffer; `out` points to the first full position in the buffer. The buffer is empty when `in == out`; the buffer is full when `((in + 1) % BUFFER_SIZE) == out`.





Producer Process – Shared Memory

```
item next_produced;

while (true) {
     /* produce an item in next_produced */

     while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */

     buffer[in] = next_produced;
     in = (in + 1) % BUFFER_SIZE;
}
```

Figure 3.12 The producer process using shared memory.

The producer process generates items and stores them in a shared bounded buffer implemented as a circular array. It uses the **in** pointer to track the next free position in the buffer. Before writing, it checks if the buffer is full by comparing **(in + 1) % BUFFER_SIZE** with **out**. If the buffer is full, the producer waits. Once space is available, it writes the item (**next_produced**) to **buffer[in]** and updates **in** to the next position using modulo arithmetic. This ensures the producer can continuously produce items as long as the buffer is not full.



Consumer Process – Shared Memory

```
item next_consumed;

while (true) {
      while (in == out)
         ; /* do nothing */

      next_consumed = buffer[out];
      out = (out + 1) % BUFFER_SIZE;

      /* consume the item in next_consumed */
}
```

Figure 3.13 The consumer process using shared memory.

The consumer process retrieves items from the shared bounded buffer using the **out** pointer, which points to the first full position. It checks if the buffer is empty by comparing **in** and **out**. If the buffer is empty, the consumer waits. When an item is available, it reads the item (**next_consumed**) from **buffer[out]** and updates **out** to the next position using modulo arithmetic. This allows the consumer to process items continuously as long as the buffer is not empty, ensuring synchronization with the producer.



Producer-Consumer Problem

Term	Definition	Usage in Code
Producer Process	A process that generates data or items to be consumed by another process.	The producer process produces items and stores them in the shared buffer (`buffer[in] = next_produced`).
Consumer Process	A process that consumes or uses data produced by another process.	The consumer process consumes items from the shared buffer (`next_consumed = buffer[out]`).
Shared Memory	A region of memory shared between processes for communication.	The `buffer`, `in`, and `out` variables reside in shared memory, allowing producer-consumer communication.
Circular Array	A data structure that wraps around when the end is reached, is used for buffers.	The `in` and `out` pointers wrap around using modulo arithmetic (`((in + 1) % BUFFER_SIZE)`).
Synchronization	Ensuring that processes access shared resources in a coordinated manner.	The producer waits if the buffer is full (`while (((in + 1) % BUFFER_SIZE) == out)`).
in Pointer	Points to the next free position in the buffer for the producer.	The producer writes to `buffer[in]` and updates `in = (in + 1) % BUFFER_SIZE`.
out Pointer	Points to the first full position in the buffer for the consumer.	The consumer reads from `buffer[out]` and updates `out = (out + 1) % BUFFER_SIZE`.
Race Condition	A situation where concurrent access to shared data leads to inconsistent results.	The code does not address synchronization, which is required to prevent race conditions.





Examples of IPC Systems - POSIX

- POSIX Shared Memory
  - Process first creates shared memory segment
    `shm_fd = shm_open(name, O CREAT | O RDWR, 0666);`
  - Also used to open an existing segment
  - Set the size of the object

    `ftruncate(shm_fd, 4096);`
  - Use **`mmap()`** to memory-map a file pointer to the shared memory object
  - Reading and writing to shared memory is done by using the pointer returned by **`mmap()`**.





Examples of IPC Systems - POSIX

Process 1 (Creator/Writer):

1. **Create Shared Memory**:
   - Use `shm_open()` with `O_CREAT | O_RDWR` to create/open a shared memory object.
   - Example: `int fd = shm_open("/my_shm", O_CREAT | O_RDWR, 0666);`
2. **Resize Shared Memory**:
   - Use `ftruncate()` to set the size of the shared memory.
   - Example: `ftruncate(fd, SIZE);`
3. **Map Shared Memory**:
   - Use `mmap()` to map the shared memory into the process's address space.
   - Example: `void *ptr = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);`
4. **Write Data**:
   - Write data to the mapped memory.
   - Example: `sprintf((char *)ptr, "Hello, shared memory!");`
5. **Keep Running**:
   - Keep the process running or use synchronization (e.g., semaphores) to ensure the reader process can access the data.



Examples of IPC Systems - POSIX

- **Process 2 (Reader):**

1. **Open Shared Memory**:
   - Use `shm_open()` with `O_RDWR` to open the existing shared memory object.
   - Example: `int fd = shm_open("/my_shm", O_RDWR, 0666);`
2. **Map Shared Memory**:
   - Use `mmap()` to map the shared memory into the process's address space.
   - Example: `void *ptr = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);`
3. **Read Data**:
   - Read data from the mapped memory.
   - Example: `printf("Data: %s\n", (char *)ptr);`
4. **Cleanup**:
   - Use `munmap()` to unmap the memory.
   - Example: `munmap(ptr, SIZE);`
   - Use `shm_unlink()` to remove the shared memory object (optional, typically done by the creator).
   - Example: `shm_unlink("/my_shm");`



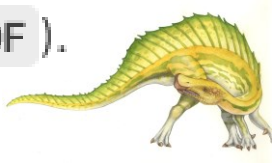


`shm_open()`:

- **Purpose**: Creates/opens a POSIX shared memory object.
- **Parameters**:
  - `const char *name`: Name of the shared memory object.
  - `int oflag`: Flags (e.g., `O_CREAT`, `O_RDWR`).
  - `mode_t mode`: Permissions (if creating).
- **Error Codes**: Returns `-1` on failure, sets `errno` (e.g., `EEXIST`, `ENOENT`).

`ftruncate()`:

- **Purpose**: Resizes the shared memory object.
- **Parameters**:
  - `int fd`: File descriptor from `shm_open()`.
  - `off_t length`: New size of the shared memory.
- **Error Codes**: Returns `-1` on failure, sets `errno` (e.g., `EINVAL`, `EBADF`).





`mmap()`:

- **Purpose**: Maps the shared memory object into the process's address space.
- **Parameters**:
  - `void *addr`: Suggested address (often `NULL`).
  - `size_t length`: Length to map.
  - `int prot`: Protection (e.g., `PROT_READ`, `PROT_WRITE`).
  - `int flags`: Flags (e.g., `MAP_SHARED`).
  - `int fd`: File descriptor from `shm_open()`.
  - `off_t offset`: Offset within the shared memory.
- **Error Codes**: Returns `MAP_FAILED` on failure, sets `errno` (e.g., `EACCES`, `ENOMEM`).

`shm_unlink()`:

- **Purpose**: Removes a POSIX shared memory object.
- **Parameters**:
  - `const char *name`: Name of the shared memory object.
- **Error Codes**: Returns `-1` on failure, sets `errno` (e.g., `ENOENT`, `EACCES`).





IPC POSIX Producer

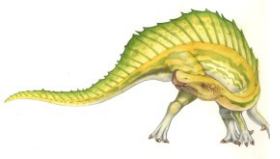
```
gcc test.c -o test -lrt
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <unistd.h>

int main() {
    const int SIZE = 4096;          // the size (in bytes) of shared memory object
    const char *name = "OS";        // name of the shared memory object
    const char *message_0 = "Hello"; // strings written to shared memory
    const char *message_1 = "World!";
    int fd;    // shared memory file descriptor
    char *ptr; // pointer to shared memory obect

    fd = shm_open(name, O_CREAT | O_RDWR, 0666); // create the shared memory object
    ftruncate(fd, SIZE); // configure the size of the shared memory object
    // memory map the shared memory object
    ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    // write to the shared memory object
    sprintf(ptr, "%s", message_0);
    ptr += strlen(message_0);
    sprintf(ptr, "%s", message_1);
    ptr += strlen(message_1);

    return 0;
}
```



Galvin and Gagne ©2018



IPC POSIX Consumer

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/mman.h>
int main() {
    const int SIZE = 4096;
    const char *name = "OS";
    int fd;
    char *ptr;
    fd = shm_open(name, O_RDONLY, 0666);
    ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    printf("%s", (char *)ptr); // read from the shared memory object
    shm_unlink(name);          // remove the shared memory object
    return 0;
}
```





IPC POSIX Consumer

```
 1  #include <stdio.h>
 2  #include <stdlib.h>
 3  #include <fcntl.h>
 4  #include <sys/shm.h>
 5  #include <sys/stat.h>
 6  #include <sys/mman.h>
 7  int main() {
 8      const int SIZE = 4096;
 9      const char *name = "OS";
10      int fd;
11      char *ptr;
12      fd = shm_open(name, O_RDONLY, 0666);
13      ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
14      printf("%s", (char *)ptr); // read from the shared memory object
```

Exception has occurred. ×
Segmentation fault



```
15      shm_unlink(name);          // remove the shared memory object
16      return 0;
17  }
```



Pipes

- A pipe acts as a conduit allowing two processes to communicate.
- They provide one of the simpler, lightweight and **synchronized** way for processes to communicate with one another for specific use cases.
- Pipes are a **unidirectional communication channel** used for Inter-process Communication (IPC) in Linux. i.e. they allow data to flow from one process to another, with one process writing to the pipe and the other reading from it.

When to Use Pipes Over Shared Memory or Message Passing?

- **Use Pipes**:
  - For simple, unidirectional data flow.
  - When processes are related (e.g., parent-child).
  - For streaming or continuous data.
- **Use Shared Memory**:
  - For high-speed, bidirectional communication.
  - When processes need to share large amounts of data.
- **Use Message Passing**:
  - For structured, discrete messages.
  - When processes are unrelated and need reliable communication.

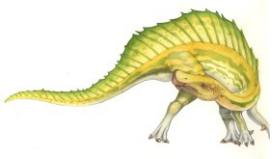




Pipes

Why Are Pipes Needed for IPC?

1. **Simplicity**:
   - Pipes are easy to set up and use compared to shared memory or message passing.
   - Example: `pipe()` creates a pipe in one line, while shared memory requires `shm_open()`, `mmap()`, etc.
2. **Synchronization**:
   - Pipes automatically handle synchronization:
     - **Blocking Reads**: A read blocks if the pipe is empty.
     - **Blocking Writes**: A write blocks if the pipe is full.
   - No need for explicit synchronization mechanisms like semaphores (required in shared memory).
3. **No Shared State**:
   - Pipes avoid the complexity of managing shared memory regions.
   - Shared memory requires careful coordination to prevent race conditions.
4. **Data Streaming**:
   - Pipes are ideal for **streaming data** (e.g., shell commands like `ls | grep "file"`).
   - Shared memory is better for fixed-size data, and message passing is more structured.
5. **Security**:
   - Pipes are isolated within processes, reducing risks of data corruption or unauthorized access.
   - Shared memory requires explicit permission management.
6. **Lightweight**:
   - Pipes have a lower overhead compared to shared memory or message queues.
   - No need for complex setup or kernel-managed structures.





Pipes

In implementing a pipe, four issues must be considered:

1. Does the pipe allow bidirectional communication, or is communication unidirectional?
2. If two-way communication is allowed, is it half duplex (data can travel only one way at a time) or full duplex (data can travel in both directions at the same time)?
3. Must a relationship (such as *parent–child*) exist between the communicating processes?
4. Can the pipes communicate over a network, or must the communicating processes reside on the same machine?

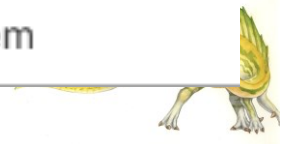
1. **Does the pipe allow bidirectional communication, or is communication unidirectional?**
   - **Unidirectional.**
2. **If two-way communication is allowed, is it half duplex or full duplex?**
   - **Half duplex** (requires two pipes for bidirectional communication).
3. **Must a relationship (such as parent–child) exist between the communicating processes?**
   - **Yes** for unnamed pipes; **No** for named pipes (FIFOs).
4. **Can the pipes communicate over a network, or must the communicating processes reside on the same machine?**
   - **Same machine only** (pipes are local IPC mechanisms).





Ordinary vs named Pipes

Feature	Ordinary (Unnamed) Pipes	Named Pipes (FIFOs)
Creation	Created with `pipe(int fd[2])`	Created with `mkfifo()`
Direction	Unidirectional	Unidirectional
Process Relationship	Requires parent-child relationship	No relationship required
Persistence	Exists only while processes are running	Persists in filesystem until deleted
Access	Accessed via file descriptors	Accessed via file path
Use Case	Communication between related processes	Communication between unrelated processes
Network Usage	Local only	Local only
Implementation	Kernel-managed circular buffer	Special file in filesystem





Pipes

Pipes in Linux for Interprocess Communication (IPC):

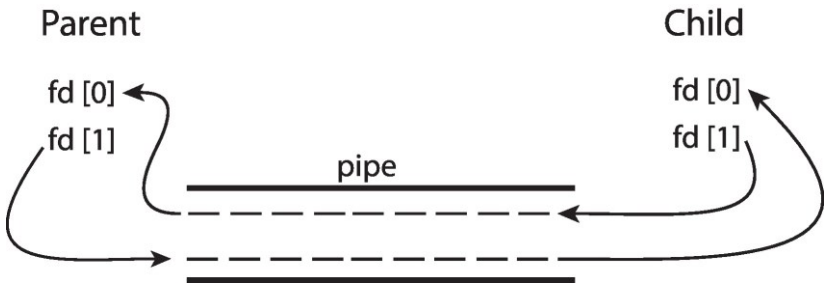
1. **Pipe Creation**:
   - Use `pipe(int fd[2])` to create a pipe.
   - `fd[0]` is the read end, `fd[1]` is the write end.
2. **Communication**:
   - **Parent-Child**: After `fork()`, one process writes to `fd[1]`, the other reads from `fd[0]`.
   - **Unrelated Processes**: Use named pipes (`mkfifo`) for communication.
3. **Writing Data**:
   - Use `write(fd[1], buffer, size)` to send data.
4. **Reading Data**:
   - Use `read(fd[0], buffer, size)` to receive data.
5. **Closing Pipes**:
   - Close unused ends with `close(fd[0])` or `close(fd[1])`.
6. **Synchronization**:
   - Pipes block on read if empty and block on write if full, ensuring synchronization.



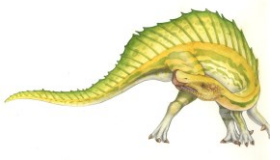


Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- **Ordinary pipes are therefore unidirectional**
- Require parent-child relationship between communicating processes



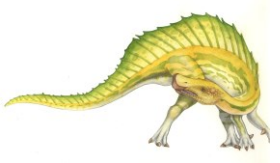
- Windows calls these **anonymous pipes**



Ordinary Pipes

```c
#include <sys/types.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#define BUFFER_SIZE 25
#define READ_END 0
#define WRITE_END 1
int main(void) {
    char write_msg[BUFFER_SIZE] = "Greetings";
    char read_msg[BUFFER_SIZE];
    int fd[2];
    pid_t pid;

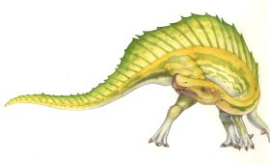
    if (pipe(fd) == -1) { // create a pipe
        fprintf(stderr, "Pipe failed");
        return 1;
    }
    pid = fork();
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    if (pid > 0) { /* parent process */
        close(fd[READ_END]);
        write(fd[WRITE_END], write_msg, strlen(write_msg) + 1);
        close(fd[WRITE_END]);
    }
    else { /* child process */
        close(fd[WRITE_END]); // close the unused end of the pipe
        read(fd[READ_END], read_msg, BUFFER_SIZE); // read from the pipe
        printf("read %s", read_msg);
        close(fd[READ_END]); // close the read end of the pipe
    }
    return 0;
}
```





Named Pipes

- A named pipe (FIFO) is a persistent, filesystem-based IPC mechanism that allows processes to communicate **bidirectionally** (half-duplex i.e., you need two FIFOs for full two-way communication).
- Created with `mkfifo()`, manipulated using `open()`, `read()`, `write()`, and `close()`. It appears as a file in the filesystem and persists until explicitly deleted.
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication. Multiple processes can write to the same named pipe.
- Processes must reside on the same machine; for network communication, use sockets.





Named Pipes (FIFOs)

1. **Persistence**: They are stored as files in the file system and can be accessed by any process as long as they have the necessary permissions. Unrelated processes can communicate using them.
2. **Bidirectional Communication**: Multiple processes can write to and read from the same named pipe. Pipes are unidirectional.
3. **Synchronization**: Named pipes provide a natural synchronization mechanism. Processes can block when reading from an empty pipe or writing to a full pipe, allowing them to synchronize their activities effectively.
4. **Multiple Readers/Writers**: Named pipes allow multiple processes to read from or write to the pipe simultaneously. This feature enables more flexible communication patterns and can be useful in scenarios where multiple processes need to access the same data concurrently.



Named Pipes

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <string.h>

#define FIFO_FILE "/tmp/myfifo"

int main() {
    int fd;
    char buffer[BUFSIZ];
    ssize_t num_bytes;

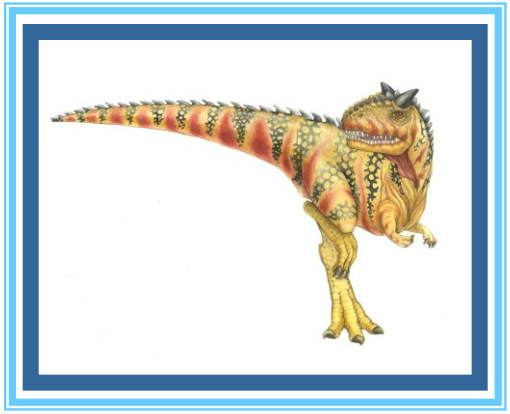
    mkfifo(FIFO_FILE, 0666);        // Create the named pipe (FIFO)
    fd = open(FIFO_FILE, O_WRONLY); // Open the named pipe for writing (producer)
    if (fd == -1) {
        perror("open");
        exit(EXIT_FAILURE);
    }
    while (1) { // Producer loop
        printf("Producer: Enter a message (or 'exit' to quit): ");
        fgets(buffer, BUFSIZ, stdin);
        num_bytes = write(fd, buffer, strlen(buffer)); // Write input to the named pipe
        if (num_bytes == -1) {
            perror("write");
            exit(EXIT_FAILURE);
        }
        if (strncmp(buffer, "exit", 4) == 0) { // Check for exit condition
            break;
        }
    }
    close(fd);          // Close the named pipe
    unlink(FIFO_FILE); // Remove the named pipe from the file system

    return 0;
}
```



ne ©2018

End of Chapter 3



Silberschatz, Galvin and Gagne ©2018